

Lab 2 - Job Scheduling System in OCaml

Group #: 3

Joey Issa - 300219818

Garrett Winslow - 300149981

Yannick Vaillancourt - 300226628

Step 1: Define Job Representation

Define a `job` type that includes three fields: `start\_time`, `duration`, and `priority`. This structure will be used to store information about each job.

```
lab2.ml ×
Lab2 > lab2.ml > ...
1  type job = {
2      start_time : int;
3      duration : int;
4      priority : int;
5  };;
6
```

Step 2: Convert Time to Minutes

Write a helper function to convert the time (hours and minutes) to total minutes from midnight. This will simplify your calculations. (name it "time_to_minutes")

```
(* converting time to minutes *)
int -> int -> int
let time_to_minutes m h = h*60+m;;
int -> int
let minutes_from_time time = time mod 60;;
int -> int
let hour_from_time time = (time - (minutes_from_time time))/60;;
int -> unit
let print_time time =
    print_int (hour_from_time time);
    print_string ":";
    Printf.printf "%02d" (minutes_from_time time);;
```

Step 3: Reading Input from the Terminal

Write a function `read\_jobs` that will prompt the user to input job details. This function should prompt for each job's start time (hours and minutes), duration (in minutes), and priority. Use `Printf.printf` to display the prompts, and `input\_line` to read input from the terminal. Make sure to trim the input to avoid errors.

```
(* Speaking of time, interrogation time *)
string -> int
let ask_int question = print_string question; read_int();;

unit -> int
let ask_time () =
  print_endline "Time (24 hour notation):";
  (time_to_minutes (ask_int "minute: ") (ask_int "hour: "));;

unit -> job
let read_job () = {
  start_time=(ask_time());
  duration=(ask_int "Job duration (minutes)?: ");
  priority=(ask_int "Job priority?: ");
};;

int -> job list -> job list
let rec read_jobs how_many cur_jobs =
  print_endline "";
  if how_many > 0 then
    read_jobs (how_many-1) (List.append [read_job()] cur_jobs)
  else
    cur_jobs;;

unit -> job list
let get_jobbin() = read_jobs (ask_int "How many jobs we doin'?:") [];;
```

Step 4: Implement Scheduling Functions

Write three functions to handle different scheduling strategies:

- `schedule\_jobs`: Ensures no job overlaps.
- `schedule\_jobs\_max\_priority`: Prioritizes jobs with higher priority.
- `schedule\_jobs\_min\_idle`: Minimizes idle time between jobs.

```
(* Comparison functions used for sorting in the different scheduling strategies *)
let compare_st_time j1 j2 = compare j1.start_time j2.start_time;;
let compare_priority j1 j2 = compare j2.priority j1.priority;;
let compare_duration j1 j2 = compare j1.duration j2.duration;;

(* Boolean function denoting if 2 jobs will overlap, creating a conflict*)
let conflict_exists j1 j2 =
  (j1.start_time <= j2.start_time && j2.start_time <= j1.start_time + j1.duration) ||
  (j2.start_time <= j1.start_time && j1.start_time <= j2.start_time + j2.duration);;

(* Boolean function denoting whether the new job conflicts with any in the schedule *)
let rec any_conflict j jlist = match jlist with
| [] -> false (* Base case meaning there's no more jobs to check for conflicts,
so return false*)
| hd :: tl -> if conflict_exists j hd then true else (any_conflict j tl);;
(* If there are jobs left, return true if there is a conflict with the head of
the job list. If not, try again with the rest of the jobs*)

(* Puts together the schedule, returning it as 'out_list'*)
let rec schedule st_sorted_jobs out_list = match st_sorted_jobs with
| [] -> out_list (*Base case: No more jobs to attempting scheduling, so return
the schedule 'out_list'*)
| hd :: tl -> if any_conflict hd out_list then
  schedule tl out_list
(*Otherwise, attempt to schedule the job. If a conflict is found, ignore the job
and try scheduling the next job*)
else schedule tl (List.append out_list [hd]);;
(* If there is no conflict, add the job to the schedule 'out_list', and proceed
to schedule the next job*)
```

```
(*These are wrappers representing the different job strategies that get
called when selected*)
job list -> job list
let no_overlap job_list =
  (schedule (List.sort compare_st_time job_list) []);;
job list -> job list
let max_priority job_list =
  (schedule (List.sort compare_priority job_list) []);;
job list -> job list
let min_idle_time job_list =
  (schedule (List.sort compare_duration job_list) []);;

(* Associates the option chosen to its corresponding scheduling strategy,
returning the completed schedule*)
job list -> int -> job list
let do_schedule_strat job_list option =
  if option = 1 then
    (no_overlap job_list)
  else if option = 2 then
    (max_priority job_list)
  else
    (min_idle_time job_list);;
```

Step 5: Print Scheduled Jobs

Implement a function `print\_schedule` to display the scheduled jobs in a user-friendly format, including their start time, duration, and priority.

```
(* Speaking of time, interrogation time *)
string -> int
let ask_int question = print_string question; read_int();;

unit -> int
let ask_time () =
  print_endline "Time (24 hour notation):";
  (time_to_minutes (ask_int "minute: ") (ask_int "hour: "));;

unit -> job
let read_job () = {
  start_time=(ask_time());
  duration=(ask_int "Job duration (minutes)? : ");
  priority=(ask_int "Job priority?: ");
};;

int -> job list -> job list
let rec read_jobs how_many cur_jobs =
  print_endline "";
  if how_many > 0 then
    read_jobs (how_many-1) (List.append [read_job()] cur_jobs)
  else
    cur_jobs;;

unit -> job list
let get_jobbin() = read_jobs (ask_int "How many jobs we doin'?:") [];;

(* Pretty printing for brainlets who cant read one big number *)
job -> unit
let print_job job = print_string "Job of priority ";
  Printf.printf "%d" job.priority;
  print_string " starting at ";
  print_time job.start_time;
  print_string " until ";
  print_time (job.start_time + job.duration);
  print_endline "";;

job list -> 'a list
let rec print_schedule jobs = match jobs with
| [] -> []
| h::ht -> print_job h; print_schedule ht;;
```

Step 6: Main Function and Terminal Input

Write a main function to bring everything together. This function should prompt the user to enter how many jobs they want to schedule, gather input for each job, and ask them to select a scheduling strategy. Then, the program should schedule and display the jobs according to the chosen strategy

```
(* Main Function *)
unit -> unit
let main () =
  let job_list = get_jobbin() in
  let option = enter_in (ask_int "Choose a scheduling strategy (1 for No Overlaps, 2 for Max Priority, 3 for Minimize Idle Time): ") [1;2;3] "Choose a scheduling strategy (1 for No Overlaps, 2 for Max Priority, 3 for Minimize Idle Time): " in
  let sorted_jobs = (do_schedule_strat job_list option) in
  print_schedule sorted_jobs;
  main();;
```

Test Cases

Base Case:

```
How many jobs we doin'?:4
Job priority?: 5
Job duration (minutes)?: 120
Time (24 hour notation):
hour: 5
minute: 0

Job priority?: 1
Job duration (minutes)?: 180
Time (24 hour notation):
hour: 9
minute: 0

Job priority?: 2
Job duration (minutes)?: 180
Time (24 hour notation):
hour: 12
minute: 0

Job priority?: 3
Job duration (minutes)?: 80
Time (24 hour notation):
hour: 6
minute: 0
```

Q1 — NO Overlaps

Output:

```
Choose a scheduling strategy (1 for No Overlaps, 2 for Max Priority, 3 for Minimize Idle Time): 1
Job of priority 5 starting at 5:00 until 7:00
Job of priority 1 starting at 9:00 until 12:20
```

Q2— Max Priority

Output

```
Choose a scheduling strategy (1 for No Overlaps, 2 for Max Priority, 3 for Minimize Idle Time): 2
Job of priority 5 starting at 5:00 until 7:00
Job of priority 2 starting at 12:00 until 15:00
```

Q3 — Minimise Idle Time

Output:

```
Choose a scheduling strategy (1 for No Overlaps, 2 for Max Priority, 3 for Minimize Idle Time): 3
Job of priority 3 starting at 6:00 until 7:20
Job of priority 2 starting at 12:00 until 15:00
```